

In-Class Assignment: Build an Idempotent Workflow Worker with SQS and DynamoDB

Goal

Build the consumer side of the workflow system.

The worker should:

1. Poll messages from SQS.
2. Read workflow_run_id from the message.
3. Load the workflow run from DynamoDB.
4. Use a conditional update to claim the workflow run.
5. Execute the correct workflow handler.
6. Update WorkflowRun.status.
7. Update Ticket.status.
8. Delete the SQS message after processing.

Final flow:

SQS Message

```
{  
  "workflow_run_id": "run_xxxxxxx"  
}
```

|

v

Workflow Worker

|

v

Conditional claim:

WorkflowRun.status QUEUED -> RUNNING

|

v

Run workflow handler

|

v

WorkflowRun.status RUNNING -> COMPLETED

Ticket.status WORKFLOW_RUNNING -> RESOLVED

|

v

Delete SQS message

Step 1: Review Current Producer Flow

The producer side already creates a workflow run and sends an SQS message.

Successful POST /tickets should already produce:

```
Ticket.status = WORKFLOW_QUEUED
WorkflowRun.status = QUEUED
SQS message body = {"workflow_run_id": "run_xxxxxxx"}
```

Before starting this assignment, create one ticket with:

POST /tickets

Request body:

```
{
  "user_id": "user_123",
  "message": "I did not get an alert when TSLA dropped below 200.",
  "source": "mobile_app"
}
```

Confirm the response has:

```
{
  "status": "WORKFLOW_QUEUED",
  "workflow_run_id": "run_xxxxxxx"
}
```

Step 2: Understand the Consumer Status Flow

The worker starts from this state:

```
Ticket.status = WORKFLOW_QUEUED
WorkflowRun.status = QUEUED
```

When the worker starts processing:

```
Ticket.status = WORKFLOW_RUNNING
WorkflowRun.status = RUNNING
```

When the workflow succeeds:

```
Ticket.status = RESOLVED
WorkflowRun.status = COMPLETED
```

When the workflow fails:

```
Ticket.status = FAILED
WorkflowRun.status = FAILED
```

Step 3: Why We Need Idempotency

SQS Standard Queue may deliver the same message more than once.

So the worker cannot directly execute the workflow after receiving a message.

Bad flow:

```
workflow_run_id = message["workflow_run_id"]
run_workflow(workflow_run_id)
```

Correct flow:

```
workflow_run_id = message["workflow_run_id"]

claimed = mark_workflow_running_if_queued(workflow_run_id)
```

#duplicate queued, tried queued 区分

```
receive_count = int(
    message.get("Attributes", {}).get("ApproximateReceiveCount", "1")
)
```

#only removed duplicate queued message

```
if not claimed and count == 1:
    delete_message()
    return
```

```
run_workflow(workflow_run_id)
```

The key idea:

Only one worker should be able to change WorkflowRun.status from QUEUED to RUNNING.

Step 4: Add New Transaction Methods

Open:

app/repositories/workflow_transaction_repository.py

Add these methods inside WorkflowTransactionRepository.

```
from botocore.exceptions import ClientError
```

```
from app.models.ticket_models import TicketStatus
```

```
from app.models.workflow_run_models import WorkflowRunStatus
```

Then add:

```
def mark_workflow_running_if_queued(
    self,
    ticket_id: str,
    workflow_run_id: str,
    updated_at: str,
) -> bool:
    try:
        self.client.transact_write_items(
            TransactItems=[
                {
                    "Update": {
                        "TableName": WORKFLOW_RUNS_TABLE_NAME,
                        "Key": self._serialize_item(
                            {
                                "workflow_run_id": workflow_run_id
                            }
                        ),
                        "UpdateExpression": (
                            "SET #status = :running, updated_at = :updated_at"
                        ),
                        "ConditionExpression": "#status = :queued",
                        "ExpressionAttributeNames": {
                            "#status": "status"
                        },
                        "ExpressionAttributeValues": self._serialize_item(
                            {
                                ":running": WorkflowRunStatus.RUNNING.value,
                                ":queued": WorkflowRunStatus.QUEUED.value,
                                ":updated_at": updated_at,
                            }
                        ),
                    }
                },
                {
                    "Update": {
                        "TableName": TICKETS_TABLE_NAME,
                        "Key": self._serialize_item(
                            {
```

```

        "ticket_id": ticket_id
    },
    "UpdateExpression": (
        "SET #status = :status, updated_at = :updated_at"
    ),
    "ConditionExpression": "attribute_exists(ticket_id)",
    "ExpressionAttributeNames": {
        "#status": "status"
    },
    "ExpressionAttributeValues": self._serialize_item(
        {
            ":status": TicketStatus.WORKFLOW_RUNNING.value,
            ":updated_at": updated_at,
        }
    ),
    },
    },
    ],
)

return True

except ClientError as error:
    if error.response["Error"]["Code"] == "TransactionCanceledException":
        return False

    raise

```

Add success update:

```

def mark_workflow_completed(
    self,
    ticket_id: str,
    workflow_run_id: str,
    result: dict,
    updated_at: str,
) -> None:
    self.client.transact_write_items(
        TransactItems=[
            {
                "Update": {
                    "TableName": WORKFLOW_RUNS_TABLE_NAME,
                    "Key": self._serialize_item(
                        {
                            "workflow_run_id": workflow_run_id
                        }
                    ),
                    "UpdateExpression": (

```

```

        "SET #status = :status, "
        "#result = :result, "
        "updated_at = :updated_at"
    ),
    "ConditionExpression": "attribute_exists(workflow_run_id)"
),
    "ExpressionAttributeNames": {
        "#status": "status",
        "#result": "result",
    },
    "ExpressionAttributeValues": self._serialize_item(
        {
            ":status": WorkflowRunStatus.COMPLETED.value,
            ":result": result,
            ":updated_at": updated_at,
        }
    ),
}
},
{
    "Update": {
        "TableName": TICKETS_TABLE_NAME,
        "Key": self._serialize_item(
            {
                "ticket_id": ticket_id
            }
        ),
        "UpdateExpression": (
            "SET #status = :status, updated_at = :updated_at"
        ),
        "ConditionExpression": "attribute_exists(ticket_id)",
        "ExpressionAttributeNames": {
            "#status": "status"
        },
        "ExpressionAttributeValues": self._serialize_item(
            {
                ":status": TicketStatus.RESOLVED.value,
                ":updated_at": updated_at,
            }
        ),
    }
},
]
)

```

Add failure update:

```

def mark_workflow_failed(
    self,
    ticket_id: str,

```

```

        workflow_run_id: str,
        error_message: str,
        updated_at: str,
    ) -> None:
        self.client.transact_write_items(
            TransactItems=[
                {
                    "Update": {
                        "TableName": WORKFLOW_RUNS_TABLE_NAME,
                        "Key": self._serialize_item(
                            {
                                "workflow_run_id": workflow_run_id
                            }
                        ),
                        "UpdateExpression": (
                            "SET #status = :status, "
                            "error_message = :error_message, "
                            "updated_at = :updated_at"
                        ),
                        "ConditionExpression": "attribute_exists(workflow_run_id)",
                        "ExpressionAttributeNames": {
                            "#status": "status"
                        },
                        "ExpressionAttributeValues": self._serialize_item(
                            {
                                ":status": WorkflowRunStatus.FAILED.value,
                                "error_message": error_message,
                                "updated_at": updated_at,
                            }
                        ),
                    },
                },
            ],
        )
        {
            "Update": {
                "TableName": TICKETS_TABLE_NAME,
                "Key": self._serialize_item(
                    {
                        "ticket_id": ticket_id
                    }
                ),
                "UpdateExpression": (
                    "SET #status = :status, updated_at = :updated_at"
                ),
                "ConditionExpression": "attribute_exists(ticket_id)",
                "ExpressionAttributeNames": {
                    "#status": "status"
                },
                "ExpressionAttributeValues": self._serialize_item(
                    {

```

```

        ):status": TicketStatus.FAILED.value,
        ):updated_at": updated_at,
    },
    },
    },
    ],
)

```

Step 5: Add Result Field to WorkflowRun Model

Open:

app/models/workflow_run_models.py

Update WorkflowRunResponse:

```

from enum import Enum
from typing import Dict, Optional

from pydantic import BaseModel


class WorkflowRunStatus(str, Enum):
    CREATED = "CREATED"
    QUEUED = "QUEUED"
    RUNNING = "RUNNING"
    COMPLETED = "COMPLETED"
    FAILED = "FAILED"


class WorkflowRunResponse(BaseModel):
    workflow_run_id: str
    ticket_id: str
    workflow_id: str
    workflow_type: str
    status: WorkflowRunStatus
    workflow_input: Dict[str, str]
    created_at: str
    updated_at: str
    result: Dict[str, str] = {}
    error_message: Optional[str] = None

```

Step 6: Update WorkflowRunRepository Mapping

Open:

app/repositories/workflow_run_repository.py

In `_workflow_run_to_item`, include result:

```
def _workflow_run_to_item(self, workflow_run: WorkflowRunResponse) -> dict:
    item = {
        "workflow_run_id": workflow_run.workflow_run_id,
        "ticket_id": workflow_run.ticket_id,
        "workflow_id": workflow_run.workflow_id,
        "workflow_type": workflow_run.workflow_type,
        "status": workflow_run.status.value,
        "workflow_input": workflow_run.workflow_input,
        "created_at": workflow_run.created_at,
        "updated_at": workflow_run.updated_at,
        "result": workflow_run.result,
    }

    if workflow_run.error_message is not None:
        item["error_message"] = workflow_run.error_message

    return item
```

In `_item_to_workflow_run`, include result:

```
def _item_to_workflow_run(self, item: dict) -> WorkflowRunResponse:
    return WorkflowRunResponse(
        workflow_run_id=item["workflow_run_id"],
        ticket_id=item["ticket_id"],
        workflow_id=item["workflow_id"],
        workflow_type=item["workflow_type"],
        status=WorkflowRunStatus(item["status"]),
        workflow_input=item.get("workflow_input", {}),
        created_at=item["created_at"],
        updated_at=item["updated_at"],
        result=item.get("result", {}),
        error_message=item.get("error_message"),
    )
```

Step 7: Create Workflow Handlers

Create folder:

app/workflows

Create:

app/workflows/__init__.py

Create:

app/workflows/workflow_handlers.py

Add:

```
import time
```

```
from app.models.workflow_run_models import WorkflowRunResponse
```

```
def handle_stock_data_issue(workflow_run: WorkflowRunResponse) -> dict:
    print("Running stock data issue workflow...")
    print(f"Workflow input: {workflow_run.workflow_input}")
```

```
    time.sleep(2)
```

```
    return {
        "message": "Stock data issue workflow completed.",
        "workflow_type": workflow_run.workflow_type,
    }
```

```
def handle_watchlist_bug(workflow_run: WorkflowRunResponse) -> dict:
    print("Running watchlist bug workflow...")
    print(f"Workflow input: {workflow_run.workflow_input}")
```

```
    time.sleep(2)
```

```
    return {
        "message": "Watchlist bug workflow completed.",
        "workflow_type": workflow_run.workflow_type,
    }
```

```
def handle_price_alert_bug(workflow_run: WorkflowRunResponse) -> dict:
    print("Running price alert bug workflow...")
    print(f"Workflow input: {workflow_run.workflow_input}")
```

```
    time.sleep(2)
```

```
    return {
        "message": "Price alert bug workflow completed.",
        "workflow_type": workflow_run.workflow_type,
    }
```

```

def handle_general_product_question(workflow_run: WorkflowRunResponse) -> dict:
    print("Running general product question workflow...")
    print(f"Workflow input: {workflow_run.workflow_input}")

    time.sleep(2)

    return {
        "message": "General product question workflow completed.",
        "workflow_type": workflow_run.workflow_type,
    }

WORKFLOW_HANDLERS = {
    "STOCK_DATA_ISSUE": handle_stock_data_issue,
    "WATCHLIST_BUG": handle_watchlist_bug,
    "PRICE_ALERT_BUG": handle_price_alert_bug,
    "GENERAL_PRODUCT_QUESTION": handle_general_product_question,
}

def run_workflow_handler(workflow_run: WorkflowRunResponse) -> dict:
    handler = WORKFLOW_HANDLERS.get(workflow_run.workflow_type)

    if handler is None:
        raise ValueError(f"No handler found for workflow type: {workflow_run.workflow_type}")

    return handler(workflow_run)

```

Step 8: Add SQS Receive and Delete Methods

Open:

app/services/sqs_service.py

Make sure it contains:

```

import json

import boto3

from app.config import AWS_REGION, WORKFLOW_QUEUE_URL

```

```

class SQSService:
    def __init__(self):
        self.sqs = boto3.client("sqs", region_name=AWS_REGION)
        self.queue_url = WORKFLOW_QUEUE_URL

    def send_workflow_run_message(self, workflow_run_id: str) -> None:
        if not self.queue_url:
            raise RuntimeError("WORKFLOW_QUEUE_URL is not set.")

        message_body = {
            "workflow_run_id": workflow_run_id
        }

        self.sqs.send_message(
            QueueUrl=self.queue_url,
            MessageBody=json.dumps(message_body),
        )

    def receive_messages(self):
        if not self.queue_url:
            raise RuntimeError("WORKFLOW_QUEUE_URL is not set.")

        response = self.sqs.receive_message(
            QueueUrl=self.queue_url,
            MaxNumberOfMessages=1,
            WaitTimeSeconds=10,
        )

        return response.get("Messages", [])

    def delete_message(self, receipt_handle: str) -> None:
        self.sqs.delete_message(
            QueueUrl=self.queue_url,
            ReceiptHandle=receipt_handle,
        )

```

Step 9: Create Workflow Worker

Create folder:

app/workers

Create:

app/workers/__init__.py

Create:

app/workers/workflow_worker.py

Add:

```
import json
from datetime import datetime, timezone

from app.repositories.workflow_run_repository import WorkflowRunRepository
from app.repositories.workflow_transaction_repository import (
    WorkflowTransactionRepository,
)
from app.services.sqs_service import SQSService
from app.workflows.workflow_handlers import run_workflow_handler

class WorkflowWorker:
    def __init__(self):
        self.sqs_service = SQSService()
        self.workflow_run_repository = WorkflowRunRepository()
        self.workflow_transaction_repository = WorkflowTransactionRepository()

    def run(self):
        print("Workflow worker started...")

        while True:
            messages = self.sqs_service.receive_messages()

            if not messages:
                print("No messages received.")
                continue

            for message in messages:
                self.process_message(message)

    def process_message(self, message: dict):
        receipt_handle = message["ReceiptHandle"]
        body = json.loads(message["Body"])

        workflow_run_id = body["workflow_run_id"]

        print(f"Received workflow_run_id: {workflow_run_id}")

        workflow_run = self.workflow_run_repository.get_by_id(workflow_run_id)

        if workflow_run is None:
            print(f"Workflow run not found: {workflow_run_id}")
            self.sqs_service.delete_message(receipt_handle)
```

```

        return

    claimed = self.workflow_transaction_repository.mark_workflow_running_
    if_queued(
        ticket_id=workflow_run.ticket_id,
        workflow_run_id=workflow_run.workflow_run_id,
        updated_at=self._current_time_iso(),
    )

    #mark 要改
    if not claimed:
        print(
            "Workflow run was already claimed or processed. "
            f"Skipping duplicate message: {workflow_run_id}"
        )
        self.sqs_service.delete_message(receipt_handle)
        return

    workflow_run = self.workflow_run_repository.get_by_id(workflow_run_id)

    try:
        result = run_workflow_handler(workflow_run)

        self.workflow_transaction_repository.mark_workflow_completed(
            ticket_id=workflow_run.ticket_id,
            workflow_run_id=workflow_run.workflow_run_id,
            result=result,
            updated_at=self._current_time_iso(),
        )

        self.sqs_service.delete_message(receipt_handle)

        print(f"Workflow run completed: {workflow_run_id}")

    except Exception as error:
        self.workflow_transaction_repository.mark_workflow_failed(
            ticket_id=workflow_run.ticket_id,
            workflow_run_id=workflow_run.workflow_run_id,
            error_message=str(error),
            updated_at=self._current_time_iso(),
        )
        #retry 不应该删掉
        self.sqs_service.delete_message(receipt_handle)

        print(f"Workflow run failed: {workflow_run_id}. Error: {error}")

    def _current_time_iso(self):

```

```
return datetime.now(timezone.utc).isoformat()
```

```
if __name__ == "__main__":  
    worker = WorkflowWorker()  
    worker.run()
```

Step 10: Run API Server

In terminal 1, run:

```
uvicorn app.main:app --reload
```

Open:

```
http://127.0.0.1:8000/docs
```

Step 11: Create a Ticket

Call:

POST /tickets

Request body:

```
{  
  "user_id": "user_123",  
  "message": "I did not get an alert when TSLA dropped below 200.",  
  "source": "mobile_app"  
}
```

Expected response:

```
{  
  "status": "WORKFLOW_QUEUED",  
  "selected_workflow_type": "PRICE_ALERT_BUG",  
  "workflow_run_id": "run_xxxxxxx"  
}
```

Copy the returned workflow_run_id.

Step 12: Verify Workflow Run Before Worker

Call:

```
GET /workflow-runs/{workflow_run_id}
```

Expected result:

```
{
  "workflow_run_id": "run_xxxxxxxx",
  "workflow_type": "PRICE_ALERT_BUG",
  "status": "QUEUED"
}
```

Step 13: Run Workflow Worker

In terminal 2, run:

```
python -m app.workers.workflow_worker
```

Expected worker logs:

```
Workflow worker started...
Received workflow_run_id: run_xxxxxxxx
Running price alert bug workflow...
Workflow run completed: run_xxxxxxxx
```

Step 14: Verify Workflow Run After Worker

Call:

```
GET /workflow-runs/{workflow_run_id}
```

Expected result:

```
{
  "workflow_run_id": "run_xxxxxxxx",
  "workflow_type": "PRICE_ALERT_BUG",
  "status": "COMPLETED",
  "result": {
    "message": "Price alert bug workflow completed.",
    "workflow_type": "PRICE_ALERT_BUG"
  }
}
```

Step 15: Verify Ticket After Worker

Call:

```
GET /tickets/{ticket_id}
```

Expected result:

```
{  
  "ticket_id": "ticket_xxxxxxx",  
  "status": "RESOLVED",  
  "workflow_run_id": "run_xxxxxxx"  
}
```

Step 16: Test Duplicate Message Handling

Send the same workflow_run_id to SQS again.

Use AWS Console:

SQS

- ticket-workflow-queue
- Send and receive messages
- Send message

Message body:

```
{  
  "workflow_run_id": "run_xxxxxxx"  
}
```

Run the worker again.

Expected worker behavior:

```
Received workflow_run_id: run_xxxxxxx  
Workflow run was already claimed or processed. Skipping duplicate message.
```

Expected final state:

```
WorkflowRun.status = COMPLETED  
Ticket.status = RESOLVED  
Workflow does not run again
```

Step 17: Test Different Workflow Types

Create another ticket:

```
{  
  "user_id": "user_123",  
  "message": "My watchlist disappeared this morning.",  
  "source": "mobile_app"  
}
```

Expected selected workflow:

WATCHLIST_BUG

Run the worker.

Expected final state:

```
WorkflowRun.status = COMPLETED  
Ticket.status = RESOLVED
```

Step 18: Commit Code

Run:

```
git status  
git add .  
git commit -m "Add idempotent SQS workflow worker"  
git push origin main
```

Acceptance Criteria

The assignment is complete when:

1. Worker can poll messages from SQS.
2. Worker can parse workflow_run_id.
3. Worker can load the workflow run from DynamoDB.
4. Worker uses conditional update to claim the workflow run.
5. Only workflow runs with status QUEUED can become RUNNING.
6. Duplicate messages are skipped.
7. Worker dispatches to the correct workflow handler by workflow_type.
8. Successful workflow execution updates WorkflowRun.status to COMPLETED.

9. Successful workflow execution updates `Ticket.status` to `RESOLVED`.
 10. Failed workflow execution updates `WorkflowRun.status` to `FAILED`.
 11. Failed workflow execution updates `Ticket.status` to `FAILED`.
 12. Worker deletes the SQS message after processing.
 13. Sending the same `workflow_run_id` twice does not execute the workflow twice.
-

Questions

1. Why does SQS require idempotent consumers?
2. Why does the worker claim the workflow run before executing it?
3. What does this transition mean?

`WorkflowRun.status: QUEUED -> RUNNING`

4. Why do we use conditional update for this transition?
5. What happens if two workers receive the same message?
6. Why does the SQS message only need `workflow_run_id`?
7. Why does the worker load workflow run details from DynamoDB?
8. Why do we need `WorkflowRun.status` instead of only `Ticket.status`?
9. Why does the worker delete the SQS message after processing?
10. What could happen if the worker crashes after marking the run as `RUNNING`?